

AISSAI HACKATHON

AI for Science: putting physics into learning

Introductory lecture

Hadrien Vroylandt

Octobre 2026

What AI has made possible

Problems solved in the last few years

Protein structure

3D structure from the sequence alone, close to experimental accuracy

Molecular simulation

quantum-chemistry accuracy at force-field cost

Weather forecasting

10-day global forecast in a minute, as good as the best physics models

Exoplanet detection

a network finds a planet hidden in 4 years of Kepler light curves

Materials discovery

hundreds of thousands of new crystals predicted stable

Maths & algorithms

faster matrix multiplication, new combinatorial constructions

Problems solved in the last few years

Protein structure

3D structure from the sequence alone, close to experimental accuracy

AlphaFold 2 (2020)

Molecular simulation

quantum-chemistry accuracy at force-field cost

Deep Potential (2018)
MACE (2022)

Weather forecasting

10-day global forecast in a minute, as good as the best physics models

GraphCast (2023)

Exoplanet detection

a network finds a planet hidden in 4 years of Kepler light curves

Shallue & Vanderburg (2018)

Materials discovery

hundreds of thousands of new crystals predicted stable

GNoME (2023)

Maths & algorithms

faster matrix multiplication, new combinatorial constructions

AlphaTensor (2022)
FunSearch (2023)

AlphaFold: Nature 596, 583 (2021) · DeePMD: PRL 120, 143001 (2018) · MACE: NeurIPS (2022) · GraphCast: Science 382, 1416 (2023) · Kepler-90i: AJ 155, 94 (2018) · GNoME: Nature 624, 80 (2023) · AlphaTensor: Nature 610, 47 (2022) · FunSearch: Nature 625, 468 (2024)

What did AI actually improve?

Speed

an expensive calculation becomes cheap

Molecular simulation

DFT $\rightarrow$ ML potential:
 10^3 – $10^6 \times$ faster

Weather

hours on a supercomputer $\rightarrow$ one minute on one chip

Search

explore a huge, high-dimensional space

Materials

combinatorial space of compositions and structures

Maths & algorithms

space of algorithms and programs, too large to enumerate

Unknown mapping

learn a relation no theory gives in a usable form

Protein structure

sequence $\rightarrow$ 3D
structure: seconds to hours instead of months to years of experiments

Finding signals

rare, weak patterns in huge datasets

Exoplanets

trained on $\sim 15\,000$ labelled Kepler signals, then searched new stars: found Kepler-90i, the 8th planet of its system

Why did these work?

Data

large curated
databases, or a
simulator that produces
labels on demand

A clear target

a benchmark, a skill
score, an exact checker

Known structure

symmetries, locality,
conservation laws
built into the model

Validation

blind tests, physical
checks, uncertainty

- ▶ None of these successes is a “black box trained on raw data”
- ▶ In science we rarely start from zero: we have **equations, symmetries, conservation laws**
- ▶ **Today's question: how do we put what we know into a learned model?**

Science for AI: the other direction

Concept from science	AI method
Langevin dynamics, non-equilibrium thermodynamics	→ diffusion models (2015, 2020)
Boltzmann distribution, spin models	→ energy-based models, Boltzmann machines
Monte Carlo (1953); Hybrid Monte Carlo from lattice QCD (1987)	→ MCMC and HMC for Bayesian inference
variational free energy, mean-field theory	→ variational inference, VAEs (ELBO = $-\text{free energy}$)
optimal transport (Monge, Kantorovich)	→ Wasserstein GANs, flow matching
group theory, symmetries	→ equivariant networks (Part II)

AI for science $\rightleftarrows$ science for AI: many ML ideas are physics in disguise.

ML in 5 minutes

Learning = fitting a function

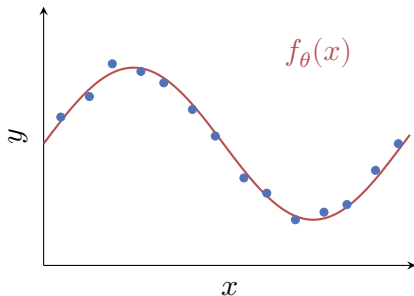
- ▶ Data: pairs (x_i, y_i) , $i = 1 \dots N$
- ▶ Model: a flexible function $f_\theta(x)$ with parameters θ
- ▶ Loss: how wrong we are

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i)$$

- ▶ Optimisation: gradient descent

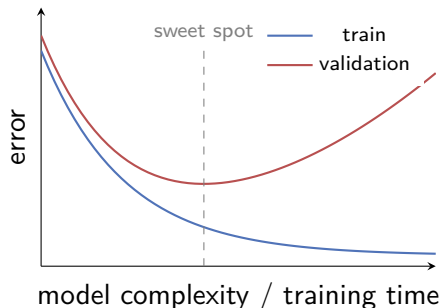
$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

- ▶ Neural network = composition of simple layers, gradients by **automatic differentiation**



Generalisation and inductive bias

- ▶ Goal: perform well on **new** data, not on the training set
- ▶ Split data: **train** / **validation** / **test**
- ▶ Overfitting: the model memorises noise
- ▶ **Inductive bias**: the assumptions built into the model decide how it generalises, especially **outside the data**
- ▶ Physics is the best inductive bias we have



Three ways to put physics into a model

I. Soft constraints

physics in the **loss**

the model is encouraged

PINNs, penalties, augmentation

II. Hard constraints

physics in the **architecture**

the model is forced

symmetries, equivariant networks,
potentials

III. Probabilistic models

learn the whole **distribution**

*the model admits what it
does not know*

generative models, uncertainty,
LLMs

What you know is data you don't need to collect.

One small demo per part (notebooks available in the hackathon repository).

I. Soft constraints: physics in the loss

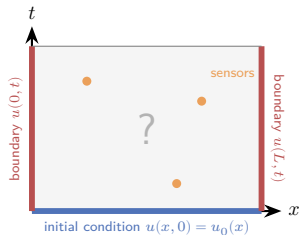
The idea: add physics to the loss

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N} \sum_i (f_\theta(x_i) - y_i)^2}_{\text{fit the data}} + \sum_k \lambda_k \underbrace{\mathcal{R}_k[f_\theta]}_{\text{violate the physics}}$$

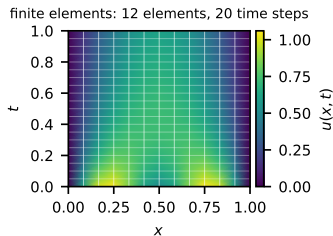
- ▶ $\mathcal{R}_k \geq 0$, zero when the physics is satisfied
- ▶ Can be evaluated **anywhere**, not only where we have data
- ▶ The network architecture does not change: any model, any optimiser
- ▶ Physics acts as a **regulariser**: it pulls the model towards physical solutions
- ▶ But the constraint is only satisfied **approximately**, as a trade-off set by λ_k

The problem: solving a PDE with a neural network

$$\partial_t u = D \partial_x^2 u \quad x \in [0, L], \quad t \in [0, T] \quad u(x, 0) = u_0(x) \quad u(0, t) = u(L, t) = 0$$

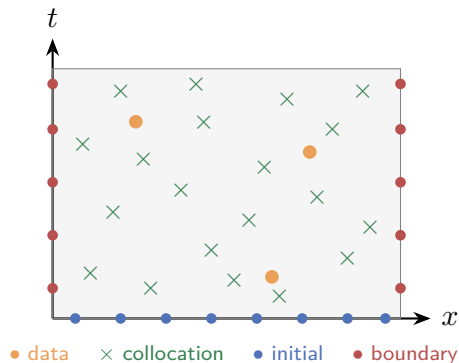


- **Known:** the equation, initial and boundary conditions, a few sensors
- **Unknown:** $u(x, t)$ everywhere inside



- Classical route (right): mesh + finite elements, solve on the grid
- Neural-network route: $u_\theta(x, t)$, mesh-free and differentiable
- **But:** no labels inside the domain. How do we train it?

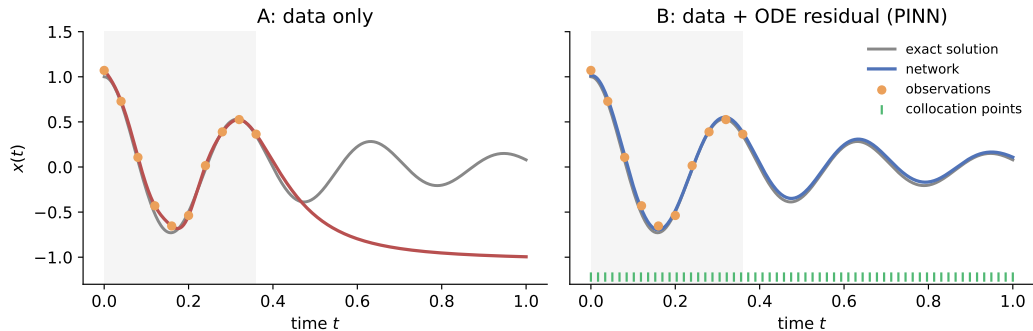
The PINN solution: the equation becomes the training signal



$$\begin{aligned}\mathcal{L}(\theta) = & \sum_i |u_\theta(x_i, t_i) - u_i|^2 \\ & + \frac{1}{N_r} \sum_r |\partial_t u_\theta - D \partial_x^2 u_\theta|_{(x_r, t_r)}^2 \\ & + \sum_j |u_\theta(x_j, 0) - u_0(x_j)|^2 \\ & + \sum_k |u_\theta(x_k^\partial, t_k) - g(t_k)|^2\end{aligned}$$

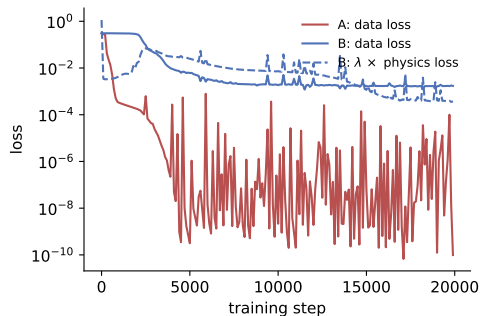
- ▶ Derivatives of u_θ by **automatic differentiation**: exact, no mesh
- ▶ Collocation points are **free**: no labels needed, put them anywhere
- ▶ No data $\Rightarrow$ forward solver; with data $\Rightarrow$ **inverse problem** (e.g. learn D too)

Demo I: a damped oscillator, observed only at the beginning

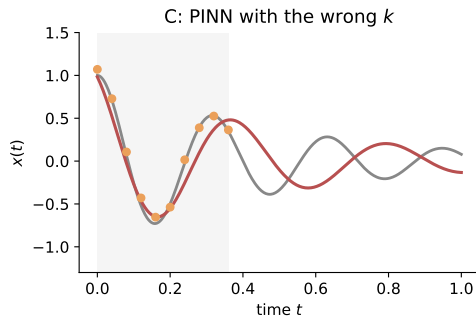


$\ddot{x} + \mu\dot{x} + kx = 0$, 10 noisy points in the grey region. **Same network**, only the loss differs: **A**: data only $\Rightarrow$ fits the points, then nonsense. **B**: + ODE residual on 60 collocation points $\Rightarrow$ follows the physics everywhere.

Demo I: two lessons



The data-only model has the **lowest training loss** (it memorises the noise) and the **worst** prediction.



A PINN with the **wrong** stiffness k : smooth, confident, and wrong. A soft constraint is only as good as the physics you put in.

Soft constraints beyond PDEs

Conservation and sum rules

- ▶ total charge $\sum_i q_i = Q$ in ML charge models
- ▶ $\int \rho(\mathbf{r}) d\mathbf{r} = N_{\text{electrons}}$, f -sum rule
- ▶ probabilities sum to one

Energy–force consistency

- ▶ separate heads for E and $\mathbf{F}$, penalty $\|\mathbf{F} + \nabla E\|^2$

Known limits

- ▶ ideal-gas limit, short-range repulsion, dissociation limit

Kinetics

- ▶ detailed balance $\pi_i k_{ij} = \pi_j k_{ji}$ in learned rate matrices or Markov state models; non-negative rates

Matching experiments

- ▶ train a potential so that simulated averages match measured ones (RDF, NMR): max-entropy reweighting, DiffTRe (Thaler & Zavadlak 2021)

Shape constraints

- ▶ monotonicity (adsorption isotherms), convexity, bounds

Any equation, inequality or limit you trust can become a loss term.

Two more soft tools

Data augmentation

- ▶ Known symmetry? Show the model transformed copies of the data (rotated, permuted, translated)
- ▶ The model **learns** the symmetry, approximately and only near the data
- ▶ The soft version of what Part II does exactly

Neural operators

- ▶ Learn a map between **functions**: initial condition, parameters $\mapsto$ solution field
- ▶ Fourier neural operators, DeepONet (Li et al. 2021; Lu et al. 2021)
- ▶ Physics-informed versions add the PDE residual to the loss: one model for a whole family of problems

Soft constraints: what goes wrong, and when to use them

What goes wrong

- ▶ Choosing the weights λ_k : competing terms
- ▶ Stiff, ill-conditioned loss landscapes; PINNs can fail to train (Krishnapriyan et al. 2021; Wang et al. 2021)
- ▶ Spectral bias: slow to learn sharp or high-frequency features
- ▶ Constraint satisfied only approximately, only where evaluated
- ▶ Wrong physics $\Rightarrow$ confident wrong model

When to use them

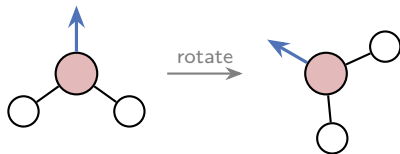
- ▶ Known equations, scarce or noisy data
- ▶ Inverse problems: infer parameters or a missing term
- ▶ Approximate physics you don't want to impose exactly
- ▶ Quick to try: only the loss changes

This week: “*What accuracy is lost by removing the physics?*” is one of the questions of the surrogate challenge.

II. Hard constraints: physics in the architecture

Symmetries: invariance and equivariance

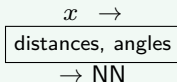
- ▶ Physical laws do not depend on how you label or orient things
 - translations, rotations, permutations of identical particles, time reversal, gauge
- ▶ **Invariance:** $f(g \cdot x) = f(x)$
 - energy $E(R\mathbf{r}) = E(\mathbf{r})$
- ▶ **Equivariance:** $f(g \cdot x) = g \cdot f(x)$
 - forces $\mathbf{F}(R\mathbf{r}) = R \mathbf{F}(\mathbf{r})$
- ▶ How do we get a model that satisfies this exactly?



E unchanged (invariant)
force rotates (equivariant)

Four routes to an exact symmetry

1. Invariant inputs

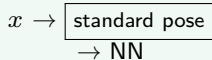


describe the system
only by invariants
(symmetry functions,
SOAP, ACE)

+ simple, cheap

– may lose information
(chirality), scalar
outputs

2. Canonicalise



align to inertia
frame, sort atoms;
or learn the frame

any network, exact

discontinuous at
degeneracies

3. Average

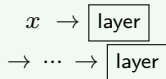
$$\bar{f}(x) = \frac{1}{|G|} \sum_g g^{-1} f(gx)$$

symmetrise any
model over the group
or a set of frames

any network, exact

cost $\propto$ number of group
elements

4. Equivariant layers

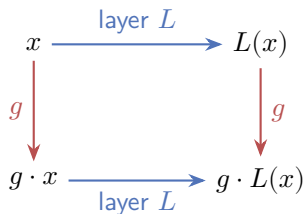


every layer commutes
with the group
(CNN, GNN, e3nn)

most expressive,
data-efficient

complex, costly

Equivariance is a constraint on the architecture



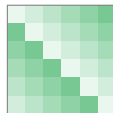
Transform then apply = apply then transform.

- ▶ Impose it **layer by layer**: equivariant layers compose into an equivariant network
- ▶ Finish with a sum or mean $\Rightarrow$ invariant output
- ▶ The constraint acts on the **weights**: only a subspace of weight matrices is allowed

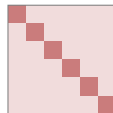
linear layer
 $L(x) = W \cdot x$
(n inputs)



no symmetry
 n^2 free weights



translations
 n weights: a **convolution**

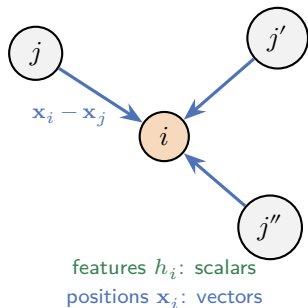


permutations
2 weights: $W = a I + b \mathbf{1}\mathbf{1}^\top$

Which symmetry, which architecture?

Symmetry	Typical data	Architecture	What the constraint does
translations	images, grids, time series	CNN	same filter everywhere
permutations (sets)	particles, point clouds	Deep Sets	same function per element, then sum
permutations (graphs)	molecules, meshes	GNN	same message on every edge
E(3): rotations + translations	atoms, 3D point clouds	EGNN, NequIP, MACE	vector/tensor features, tensor products
rotations on the sphere	climate, cosmology maps	spherical CNN	convolution on the sphere
Lorentz group	particle jets	Lorentz-equivariant	four-vectors, Minkowski products
gauge symmetry	lattice field theory	gauge-equivariant	built from gauge-invariant loops

Example: an E(3)-equivariant graph network (EGNN)



$$m_{ij} = \phi_e(h_i, h_j, \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \sum_j (\mathbf{x}_i - \mathbf{x}_j) \phi_x(m_{ij})$$

$$h_i \leftarrow \phi_h(h_i, \sum_j m_{ij})$$

- ▶ ϕ_e, ϕ_x, ϕ_h : ordinary MLPs, **unconstrained**
- ▶ They only see **distances** $\Rightarrow h$ is **invariant**
- ▶ Positions move along **relative vectors** times scalars $\Rightarrow \mathbf{x}$ is **equivariant**
- ▶ Differences $\Rightarrow$ translations drop out; sums over $j \Rightarrow$ atom order does not matter

The MLPs are free; **the way they are wired** guarantees the symmetry.
NequIP and MACE extend this to higher-order tensor features.

Hard constraints beyond symmetry

Route	Example
derive from a potential	$\mathbf{F} = -\nabla E_\theta$: conservative forces; $\mathbf{B} = \nabla \times \mathbf{A}_\theta$: divergence-free; Hamiltonian / Lagrangian NNs conserve energy
output parametrisation	softplus $\Rightarrow$ positive; softmax $\Rightarrow$ normalised; $LL^\top \Rightarrow$ positive-definite matrix; input-convex NNs $\Rightarrow$ convex
projection layer	$q_i \leftarrow q_i - (\sum_j q_j - Q)/N$: exact total charge; optimisation layers (OptNet)
trial-function ansatz	$u = g(x) + d(x) \text{NN}(x)$: boundary conditions exact (Lagaris et al. 1998)
locality, extensivity	$E = \sum_i \varepsilon(\text{environment}_i)$: size-extensive, linear scaling
hybrid model	keep the known solver, learn only the missing term (universal differential equations)
normalised by construction	normalizing flows: exact probability densities ($\rightarrow$ Part III)

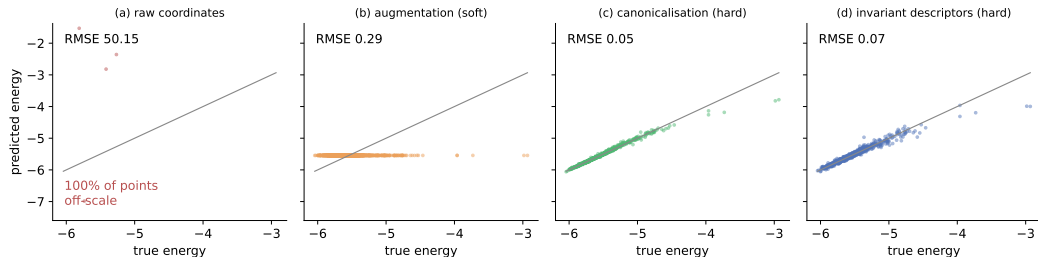
Soft vs hard

	Soft (Part I)	Hard (Part II)
how	penalty in the loss, augmentation	built into the model
guarantee	approximate, near the data	exact, everywhere
data needed	more	less
flexibility	any architecture	needs a suitable design
if the physics is wrong	model can compromise	model cannot escape

Next: the demo puts **no constraint**, **soft** (augmentation) and **hard** (canonicalisation, invariant descriptors) side by side on the same problem.

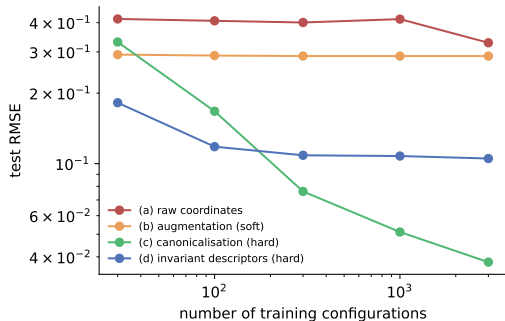
Demo II: four ways to handle symmetry

Energy of a 6-atom cluster (Lennard-Jones + three-body term). Trained in **one orientation**, tested on **rotated and permuted** copies.

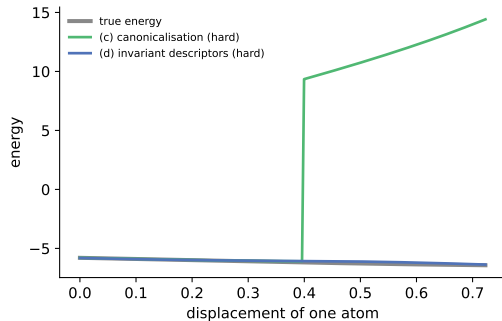


(a) raw coordinates: off-scale on rotated data. (b) augmentation: nearly invariant... by predicting $\approx$ the mean energy. (c), (d): $E(x) = E(gx)$ to 10^{-14} , by construction.

Demo II: data efficiency and the price of each route



Training data now in random orientations.
Hard constraints learn with far less data.
Invariant descriptors win with little data but plateau: **they lose information**.

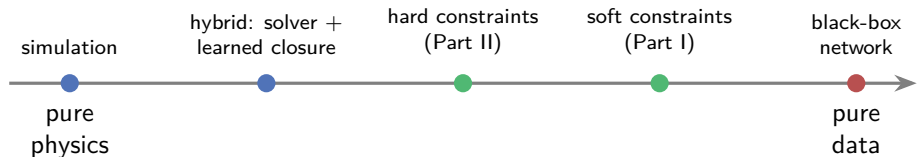


Move one atom smoothly: two atoms swap places in the sorting order and the canonicalised model **jumps**. Exact symmetry, but a discontinuous model.

Hard constraints: what goes wrong

- ▶ You must know the constraint **exactly**
- ▶ **Broken** symmetries: external fields, boundaries, finite-size effects
- ▶ Canonicalisation: discontinuities at degenerate configurations
- ▶ Invariant descriptors: information loss
- ▶ Expressivity and computational cost of equivariant layers
- ▶ **Over-constraining**: no room left to learn what the physics misses

Where we stand: a spectrum



- ▶ The further to the right, the more data you need and the less you can extrapolate
- ▶ Most successful AI-for-science models sit in the **middle**
- ▶ So far, every model returns **one number**. How much should we trust it? → Part III

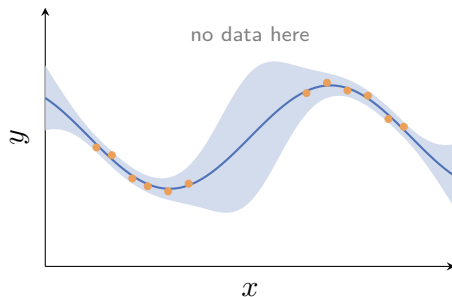
III. Generative and probabilistic models

From a value to a distribution

- ▶ Predict $p(y | x)$ instead of y
- ▶ Minimising the MSE = maximising a Gaussian likelihood with fixed width
- ▶ Let the network also predict the width:

$$-\log p = \frac{(y - \mu_{\theta}(x))^2}{2\sigma_{\theta}(x)^2} + \log \sigma_{\theta}(x)$$

- ▶ The model now says **how sure** it is



Two kinds of uncertainty

Aleatoric: noise in the data

- ▶ measurement noise, thermal fluctuations, chaotic dynamics
- ▶ does **not** go away with more data
- ▶ captured by predicting $\sigma_{\theta}(x)$

Epistemic: lack of data

- ▶ the model has not seen this region
- ▶ **decreases** with more data
- ▶ Gaussian processes, deep ensembles (Lakshminarayanan et al. 2017), MC dropout, Bayesian NNs

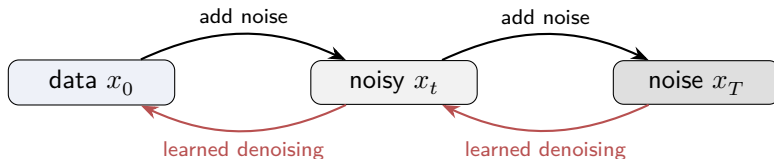
Why it matters

- ▶ Trust: is the prediction usable?
- ▶ Out-of-distribution warning
- ▶ Active learning: compute or measure where the model is unsure
- ▶ Bayesian optimisation: choose where to look next

Calibration: a 90% interval should contain the truth 90% of the time. A neural network can be **confidently wrong** far from the data, and uncertainty estimates can be miscalibrated too.

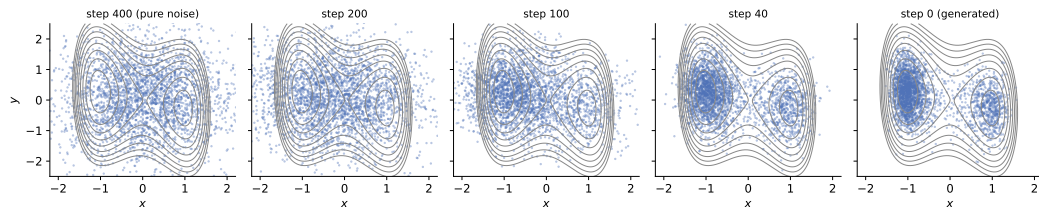
Learning the whole distribution $p(x)$

- ▶ **Generative model:** learn $p(x)$ from samples, then **sample** new x or **score** a given x
- ▶ Variational autoencoders: encode to a small latent space, decode back
- ▶ Normalizing flows: invertible map from a Gaussian, exact $\log p(x)$
- ▶ **Diffusion models** (Ho et al. 2020): learn to undo a gradual noising
 - generation = learned, Langevin-like dynamics from noise to data



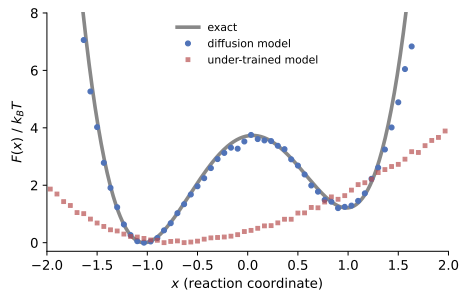
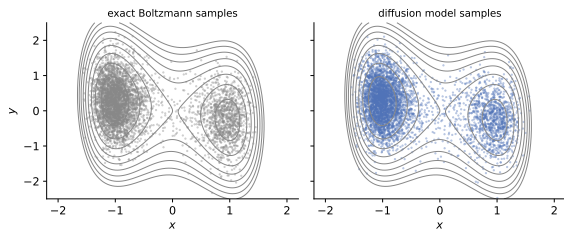
Demo III: a diffusion model learns a Boltzmann distribution

Target: $p(x, y) \propto e^{-U(x, y)/k_B T}$, a tilted double well. 50,000 exact samples, a small network that predicts the added noise.



From pure noise to the two basins, by running the learned denoising backwards.

Demo III: check the physics, not the pictures



- ▶ Free energy $F(x) = -k_B T \ln p(x)$: the trained model gets the **barrier** right
- ▶ The under-trained model gets the basin populations right (0.25 vs 0.25)... by luck: the barrier is gone
- ▶ One summary number can look fine while the distribution is wrong. The barrier is where samples are rare.

Physics meets generative models

Boltzmann generators (Noé et al. 2019)

- ▶ Flow trained to sample $p(x) \propto e^{-E(x)/k_B T}$ directly
- ▶ Uses the **energy** as a training signal (soft, Part I)
- ▶ Equivariant flows respect symmetries (hard, Part II)
- ▶ Exact likelihood $\Rightarrow$ reweighting gives **unbiased** averages (Part III)

Other uses

- ▶ Protein and molecule design (RFdiffusion)
- ▶ Inverse problems with a prior
- ▶ Probabilistic weather forecasts (GenCast)
- ▶ Fast generation of simulated events

All three parts meet here.

Probabilistic models: links and pitfalls

Links to this week

- ▶ **Anomaly detection:** low likelihood under a model of “normal” $\Rightarrow$ suspicious
- ▶ **Optimisation:** uncertainty decides where to evaluate next (Bayesian optimisation)
- ▶ **Surrogates:** error bars tell you when to stop trusting the model (out of distribution)

What goes wrong

- ▶ Evaluating generative models: “looks realistic” is not a metric
- ▶ Mode collapse; rare events under-represented
- ▶ Overconfidence out of distribution
- ▶ Uncertainty that is itself not calibrated

Conclusion

Three ways to put physics into a model

	encodes	guarantee	tools	use when
I. Soft	equations, limits, sum rules in the loss	approximate, near the evaluated points	PINNs, penalties, augmentation	physics known but approximate; few data
II. Hard	symmetries and structure in the model	exact, everywhere	invariant features, canonicalisation, equivariant nets, potentials	constraint known exactly
III. Probabilistic	the full distribution	honest error bars (if calibrated)	GPs, ensembles, flows, diffusion	you need uncertainty or samples

They combine: an equivariant (II) generative model (III) trained with an energy penalty (I).

What we did not cover

- ▶ **Data:** collection, cleaning, labelling, curation, FAIR data, leakage between train and test
- ▶ **The mathematics:** approximation theory, optimisation landscapes, why deep networks generalise
- ▶ **LLMs and agents:** literature search, coding assistants, autonomous “AI scientists”
- ▶ **Reinforcement learning and control** in depth
- ▶ **Interpretability:** what did the model learn? Symbolic regression, explainability tools
- ▶ **Scale:** foundation models for science, GPUs, energy cost
- ▶ **Deployment:** running models in production, monitoring drift

Some of it this week

- ▶ *Accelerators and AI:* Bayesian optimisation, autoencoders, surrogates, RL in practice
- ▶ *HPC and Slurm:* running on GPUs and clusters
- ▶ *Feature Store:* data and feature engineering for your challenge dataset

And in your projects: **data** will probably be the hardest part.

Resources

Software

- ▶ General: PyTorch, JAX, scikit-learn
- ▶ Physics-informed: DeepXDE, neuraloperator
- ▶ Symmetry: PyTorch Geometric, e3nn, MACE
- ▶ Uncertainty: GPyTorch, BoTorch
- ▶ Generative: sbi (simulation-based inference)

Demos of this lecture: demos/ (three notebooks, CPU, about 1 min each)

To go further

- ▶ *Physics-based Deep Learning* (Thuerey et al., online book)
- ▶ *Geometric Deep Learning* (Bronstein et al., 2021)
- ▶ *Deep Learning* (Goodfellow, Bengio, Courville)

Next: Accelerators and AI

Questions?

Have a great hackathon!